

Experiment 'p_6_2_f_move_even_to_front_array' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 5

Hard fails (resolution): 3

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_6_2_f_move_even_to_front_array

Natural language statement: Write a method to move all even elements in an array to the front, otherwise preserving the order of the elements.

Method signature: p_6_2_f_move_even_to_front_array(inputs: array<int>) returns (result: array<int>)

Ensures

- `result[..] == evens(inputs[..]) + odds(inputs[..])`

Functional Code Given

```
function evens(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 == 0
    then [s[0]] + evens(s[1..])
    else evens(s[1..])
}

function odds(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 != 0
    then [s[0]] + odds(s[1..])
    else odds(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to move all even elements in an array to the front, otherwise preserving
→ the order of the elements.

The signature should be:

`p_6_2_f_move_even_to_front_array(inputs: array<int>) returns (result: array<int>)`

The method should respect the following contract:

`ensures result[..] == evens(inputs[..]) + odds(inputs[..])`

The contract uses the following dafny code:

```
function evens(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 == 0
    then [s[0]] + evens(s[1..])
    else evens(s[1..])
}

function odds(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 != 0
    then [s[0]] + odds(s[1..])
    else odds(s[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn0z96CwfVTuMUtXa3V59GaUrbslv

Model: gpt-5.5-2026-04-23

Created at: 1780574231

Finish reason: stop

Usage: 588 tokens in, and 8730 tokens out

```

{
  var n := inputs.Length;
  var es: seq<int> := [];
  var os: seq<int> := [];
  var i := n;

  assert inputs[n..] == [];
  assert evens(inputs[n..]) == [];
  assert odds(inputs[n..]) == [];

  while 0 < i
    invariant 0 <= i <= n
    invariant es == evens(inputs[i..])
    invariant os == odds(inputs[i..])
  {
    var oldI := i;
    var k := i - 1;
    var x := inputs[k];
    ghost var tail := inputs[oldI..];
    ghost var whole := inputs[k..];

    assert 0 <= k < n;
    assert oldI == k + 1;
    assert inputs[k..oldI] == [x];
    assert whole == inputs[k..oldI] + tail;
    assert whole == [x] + tail;
    assert |whole| != 0;
    assert whole[0] == x;
    assert whole[1..] == tail;

    if x % 2 == 0 {
      es := [x] + es;
      assert evens(whole) == [x] + evens(tail);
      assert odds(whole) == odds(tail);
      assert es == evens(inputs[k..]);
      assert os == odds(inputs[k..]);
    } else {
      os := [x] + os;
      assert evens(whole) == evens(tail);
      assert odds(whole) == [x] + odds(tail);
      assert es == evens(inputs[k..]);
      assert os == odds(inputs[k..]);
    }

    i := k;
  }

  assert i == 0;
  assert inputs[i..] == inputs[..];
  assert es == evens(inputs[..]);
  assert os == odds(inputs[..]);
}

```

```

var combined := es + os;
result := new int[|combined|](j => combined[j]);

assert result.Length == |combined|;
assert forall j :: 0 <= j < |combined| ==> result[j] == combined[j];
assert |result[..]| == |combined|;
assert forall j | 0 <= j < |combined| ensures result[..][j] == combined[j] {
  assert result[..][j] == result[j];
}
assert result[..] == combined;
assert combined == evens(inputs[..]) + odds(inputs[..]);
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_6_2_f_move_even_to_front_array/program.dfy(82,0): Error: invalid
```

```

  ↳ UnaryExpression
  |
82 | }
  | ^

```

```
problem_data/p_6_2_f_move_even_to_front_array/program.dfy(81,57): Error: expected either
```

```

  ↳ 'by' or a semicolon following the assert expression
  |
81 | assert combined == evens(inputs[..]) + odds(inputs[..]);
  | ^

```

2 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the

- ↳ Dafny body, including the curly braces that surround it. Do not show the signature
- ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn129NrLlM9VEnjUwh9KdqdPOWnGe

Model: gpt-5.5-2026-04-23

Created at: 1780574417

Finish reason: stop

Usage: 1275 tokens in, and 1036 tokens out

```

{
  var n := inputs.Length;
  var es: seq<int> := [];
  var os: seq<int> := [];
  var i := n;

```

```

assert inputs[n..] == [];
assert evens(inputs[n..]) == [];
assert odds(inputs[n..]) == [];

while 0 < i
  invariant 0 <= i <= n
  invariant es == evens(inputs[i..])
  invariant os == odds(inputs[i..])
{
  var oldI := i;
  var k := i - 1;
  var x := inputs[k];
  ghost var tail := inputs[oldI..];
  ghost var whole := inputs[k..];

  assert 0 <= k < n;
  assert oldI == k + 1;
  assert inputs[k..oldI] == [x];
  assert whole == inputs[k..oldI] + tail;
  assert whole == [x] + tail;
  assert |whole| != 0;
  assert whole[0] == x;
  assert whole[1..] == tail;

  if x % 2 == 0 {
    es := [x] + es;
    assert evens(whole) == [x] + evens(tail);
    assert odds(whole) == odds(tail);
    assert es == evens(inputs[k..]);
    assert os == odds(inputs[k..]);
  } else {
    os := [x] + os;
    assert evens(whole) == evens(tail);
    assert odds(whole) == [x] + odds(tail);
    assert es == evens(inputs[k..]);
    assert os == odds(inputs[k..]);
  }

  i := k;
}

assert i == 0;
assert inputs[i..] == inputs[..];
assert es == evens(inputs[..]);
assert os == odds(inputs[..]);

var combined := es + os;
result := new int[|combined|](j => combined[j]);

assert result.Length == |combined|;
assert forall j :: 0 <= j < |combined| ==> result[j] == combined[j];

```

```

assert |result[..]| == |combined|;
assert forall j | 0 <= j < |combined| ensures result[..][j] == combined[j] {
  assert result[..][j] == result[j];
}
assert result[..] == combined;
assert combined == (evens(inputs[..]) + odds(inputs[..]));
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_6_2_f_move_even_to_front_array/program.dfy(82,0): Error: invalid
  ↳ UnaryExpression
|
82 | }
  | ^

problem_data/p_6_2_f_move_even_to_front_array/program.dfy(81,59): Error: expected either
  ↳ 'by' or a semicolon following the assert expression
|
81 | assert combined == (evens(inputs[..]) + odds(inputs[..]));
  | ^

```

2 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn12d7e7YExo5a6QpFiRAyPz1vcrD

Model: gpt-5.5-2026-04-23

Created at: 1780574447

Finish reason: stop

Usage: 2068 tokens in, and 2602 tokens out

```

{
  var n := inputs.Length;
  var es: seq<int> := [];
  var os: seq<int> := [];
  var i := n;

  assert inputs[n..] == [];
  assert evens(inputs[n..]) == [];
  assert odds(inputs[n..]) == [];

  while 0 < i
    invariant 0 <= i <= n

```

```

    invariant es == evens(inputs[i..])
    invariant os == odds(inputs[i..])
  {
    var oldI := i;
    var k := i - 1;
    var x := inputs[k];
    ghost var tail := inputs[oldI..];
    ghost var whole := inputs[k..];

    assert 0 <= k < n;
    assert oldI == k + 1;
    assert inputs[k..oldI] == [x];
    assert whole == inputs[k..oldI] + tail;
    assert whole == [x] + tail;
    assert |whole| != 0;
    assert whole[0] == x;
    assert whole[1..] == tail;

    if x % 2 == 0 {
      es := [x] + es;
      assert evens(whole) == [x] + evens(tail);
      assert odds(whole) == odds(tail);
      assert es == evens(inputs[k..]);
      assert os == odds(inputs[k..]);
    } else {
      os := [x] + os;
      assert evens(whole) == evens(tail);
      assert odds(whole) == [x] + odds(tail);
      assert es == evens(inputs[k..]);
      assert os == odds(inputs[k..]);
    }
  }

  i := k;
}

assert i == 0;
assert inputs[i..] == inputs[..];
assert es == evens(inputs[..]);
assert os == odds(inputs[..]);

var combined := es + os;
result := new int[|combined|](j => combined[j]);

assert result.Length == |combined|;
assert forall j : 0 <= j < |combined| ==> result[j] == combined[j];
assert |result[..]| == |combined|;
assert forall j | 0 <= j < |combined| ensures result[..][j] == combined[j] {
  assert result[..][j] == result[j];
}
assert result[..] == combined;
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_6_2_f_move_even_to_front_array/program.dfy(81,0): Error: invalid
```

```
  ↳ UnaryExpression
```

```
  |  
81 | }  
  | ^
```

```
problem_data/p_6_2_f_move_even_to_front_array/program.dfy(80,31): Error: expected either
```

```
  ↳ 'by' or a semicolon following the assert expression
```

```
  |  
80 | assert result[..] == combined;  
  | ^
```

2 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the

```
  ↳ Dafny body, including the curly braces that surround it. Do not show the signature
```

```
  ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn13atUTUKyIdlXphg.JAH7SFL5GBo

Model: gpt-5.5-2026-04-23

Created at: 1780574506

Finish reason: stop

Usage: 2828 tokens in, and 3170 tokens out

```
{  
  var n := inputs.Length;  
  var es: seq<int> := [];  
  var os: seq<int> := [];  
  var i := n;  
  
  assert inputs[n..] == [];  
  assert evens(inputs[n..]) == [];  
  assert odds(inputs[n..]) == [];  
  
  while 0 < i  
  invariant 0 <= i <= n  
  invariant es == evens(inputs[i..])  
  invariant os == odds(inputs[i..])  
  {  
    var oldI := i;  
    var k := i - 1;  
    var x := inputs[k];  
    ghost var tail := inputs[oldI..];  
    ghost var whole := inputs[k..];
```



```

    assert 0 <= k < n;
    assert oldI == k + 1;
    assert inputs[k..oldI] == [x];
    assert whole == inputs[k..oldI] + tail;
    assert whole == [x] + tail;
    assert |whole| != 0;
    assert whole[0] == x;
    assert whole[1..] == tail;

    if x % 2 == 0 {
        es := [x] + es;
        assert evens(whole) == [x] + evens(tail);
        assert odds(whole) == odds(tail);
        assert es == evens(inputs[k..]);
        assert os == odds(inputs[k..]);
    } else {
        os := [x] + os;
        assert evens(whole) == evens(tail);
        assert odds(whole) == [x] + odds(tail);
        assert es == evens(inputs[k..]);
        assert os == odds(inputs[k..]);
    }

    i := k;
}

assert i == 0;
assert inputs[i..] == inputs[..];
assert es == evens(inputs[..]);
assert os == odds(inputs[..]);

var combined := es + os;
result := new int[|combined|](j => combined[j]);

assert result[..] == combined by {
    assert result.Length == |combined|;
    assert |result[..]| == |combined|;
    assert forall j :: 0 <= j < |combined| ==> result[j] == combined[j];
    assert forall j :: 0 <= j < |combined| ==> result[..][j] == combined[j];
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_6_2_f_move_even_to_front_array/program.dfy(72,45): Error: index out of
  ↳ range
  |
72 | result := new int[|combined|](j => combined[j]);
   | ^

```

Dafny program verifier finished with 3 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the

- ↪ Dafny body, including the curly braces that surround it. Do not show the signature
- ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn15FSv19Dzxlhht0mhtAIqv1z7US

Model: gpt-5.5-2026-04-23

Created at: 1780574609

Finish reason: stop

Usage: 3519 tokens in, and 4528 tokens out

```
{
  var n := inputs.Length;
  var es: seq<int> := [];
  var os: seq<int> := [];
  var i := n;

  assert inputs[n..] == [];
  assert evens(inputs[n..]) == [];
  assert odds(inputs[n..]) == [];

  while 0 < i
    invariant 0 <= i <= n
    invariant es == evens(inputs[i..])
    invariant os == odds(inputs[i..])
  {
    var oldI := i;
    var k := i - 1;
    var x := inputs[k];
    ghost var tail := inputs[oldI..];
    ghost var whole := inputs[k..];

    assert 0 <= k < n;
    assert oldI == k + 1;
    assert inputs[k..oldI] == [x];
    assert whole == inputs[k..oldI] + tail;
    assert whole == [x] + tail;
    assert |whole| != 0;
    assert whole[0] == x;
    assert whole[1..] == tail;

    if x % 2 == 0 {
      es := [x] + es;
      assert evens(whole) == [x] + evens(tail);
      assert odds(whole) == odds(tail);
    }
  }
}
```

```

    assert es == evens(inputs[k..]);
    assert os == odds(inputs[k..]);
  } else {
    os := [x] + os;
    assert evens(whole) == evens(tail);
    assert odds(whole) == [x] + odds(tail);
    assert es == evens(inputs[k..]);
    assert os == odds(inputs[k..]);
  }

  i := k;
}

assert i == 0;
assert inputs[i..] == inputs[..];
assert es == evens(inputs[..]);
assert os == odds(inputs[..]);

var combined := es + os;
var m := |combined|;
result := new int[m];

var j := 0;
while j < m
  invariant 0 <= j <= m
  invariant result.Length == m
  invariant forall t :: 0 <= t < j ==> result[t] == combined[t]
{
  result[j] := combined[j];
  j := j + 1;
}

assert j == m;
assert forall t :: 0 <= t < m ==> result[t] == combined[t];
assert |result[..]| == m;
forall t | 0 <= t < m
  ensures result[..][t] == combined[t]
{
  assert result[..][t] == result[t];
}
assert forall t :: 0 <= t < |result[..]| ==> result[..][t] == combined[t];
assert result[..] == combined;
assert combined == es + os;
assert combined == evens(inputs[..]) + odds(inputs[..]);
}

```

Final Program

```

function evens(s: seq<int>): seq<int>
{

```

```

    if |s| == 0 then []
    else if s[0] % 2 == 0
        then [s[0]] + evens(s[1..])
        else evens(s[1..])
}

function odds(s: seq<int>): seq<int>
{
    if |s| == 0 then []
    else if s[0] % 2 != 0
        then [s[0]] + odds(s[1..])
        else odds(s[1..])
}

method p_6_2_f_move_even_to_front_array(inputs: array<int>) returns (result: array<int>)
    ensures result[..] == evens(inputs[..]) + odds(inputs[..])
{
    var n := inputs.Length;
    var es: seq<int> := [];
    var os: seq<int> := [];
    var i := n;

    assert inputs[n..] == [];
    assert evens(inputs[n..]) == [];
    assert odds(inputs[n..]) == [];

    while 0 < i
        invariant 0 <= i <= n
        invariant es == evens(inputs[i..])
        invariant os == odds(inputs[i..])
    {
        var oldI := i;
        var k := i - 1;
        var x := inputs[k];
        ghost var tail := inputs[oldI..];
        ghost var whole := inputs[k..];

        assert 0 <= k < n;
        assert oldI == k + 1;
        assert inputs[k..oldI] == [x];
        assert whole == inputs[k..oldI] + tail;
        assert whole == [x] + tail;
        assert |whole| != 0;
        assert whole[0] == x;
        assert whole[1..] == tail;

        if x % 2 == 0 {
            es := [x] + es;
            assert evens(whole) == [x] + evens(tail);
            assert odds(whole) == odds(tail);
            assert es == evens(inputs[k..]);

```

```

    assert os == odds(inputs[k..]);
  } else {
    os := [x] + os;
    assert evens(whole) == evens(tail);
    assert odds(whole) == [x] + odds(tail);
    assert es == evens(inputs[k..]);
    assert os == odds(inputs[k..]);
  }

  i := k;
}

assert i == 0;
assert inputs[i..] == inputs[..];
assert es == evens(inputs[..]);
assert os == odds(inputs[..]);

var combined := es + os;
var m := |combined|;
result := new int[m];

var j := 0;
while j < m
  invariant 0 <= j <= m
  invariant result.Length == m
  invariant forall t :: 0 <= t < j ==> result[t] == combined[t]
{
  result[j] := combined[j];
  j := j + 1;
}

assert j == m;
assert forall t :: 0 <= t < m ==> result[t] == combined[t];
assert |result[..]| == m;
forall t | 0 <= t < m
  ensures result[..][t] == combined[t]
{
  assert result[..][t] == result[t];
}
assert forall t :: 0 <= t < |result[..]| ==> result[..][t] == combined[t];
assert result[..] == combined;
assert combined == es + os;
assert combined == evens(inputs[..]) + odds(inputs[..]);
}

```

Total Token Usage

Input tokens: 10278

Output tokens: 20066

Reasoning tokens: 16755

Sum of ‘total tokens’: 30344

Experiment Timings

Overall Experiment started at 1780574227701, ended at 1780574724997, lasting 497296ms (497.30 seconds)

Iteration #4 started at 1780574504515, ended at 1780574608142, lasting 103627ms (103.63 seconds)

Iteration #5 started at 1780574608160, ended at 1780574724978, lasting 116818ms (116.82 seconds)

Iteration #1 started at 1780574227702, ended at 1780574416661, lasting 188959ms (188.96 seconds)

Iteration #2 started at 1780574416662, ended at 1780574446767, lasting 30105ms (30.11 seconds)

Iteration #3 started at 1780574446768, ended at 1780574504514, lasting 57746ms (57.75 seconds)